Тема 5. Програмиране на задачи с цикли. Оператори for, while, do...while. Оператори break и continue.

Цел: Съставяне, изпълнение и анализ на C++ програми с повторение, обработка на последователности, валидиране на вход, вложени цикли и итерационни изчисления.

1. Основни конструкции за цикъл

Цикълът повтаря оператор или блок от оператори, докато е изпълнено определено условие. Трите основни конструкции в C++ са for, while и do...while. В тялото на цикъла може да има други проверки или цикли.

1.1. Цикъл for — повторения с брояч

```
for (инициализация; условие; актуализация) {
    // тяло на цикъла
}
```

Инициализацията се изпълнява еднократно. Условието се проверява преди всяка итерация и цикълът продължава, докато то е true. Актуализацията се изпълнява след тялото. Ако условието е false още в началото, тялото няма да се изпълни. Пропуснатото условие се приема за true, а пропуснатите други две секции не извършват действие.

Пример: сума на целите числа от 1 до 100 в прав и обратен ред. Променливата sum се дефинира преди цикъла, за да бъде достъпна и след него.

```
int sum = 0;
for (int i = 1; i <= 100; ++i) sum += i;

sum = 0;
for (int i = 100; i >= 1; --i) sum += i;
```

1.2. Цикъл while — повторение с предусловие

```
while (условие) {
    // тяло на цикъла
}
```

Условието се проверява преди всяка итерация, затова тялото може да не се изпълни нито веднъж. Конструкцията е удобна, когато броят на повторенията не е известен предварително — например въвеждане до специална стойност или достигане на зададена точност.

```
int i = 1, sum = 0;
while (i <= 100) {
    sum += i;
    ++i;
}
```

1.3. Цикъл do...while — повторение с постусловие

```
do {
    // тяло на цикъла
} while (условие);
```

Тялото се изпълнява поне веднъж, защото условието се проверява в края. Цикълът е удобен за менюта и повторно въвеждане на данни до удовлетворяване на условие.

```
int i = 1, sum = 0;
do {
    sum += i;
    ++i;
} while (i <= 100);
```

2. Управление на повторенията: break, continue и return

Операторът break прекъсва най-близкия обхващащ цикъл (или switch) и продължава с първия оператор след него. Използва се например за спиране при въвеждане на стойност за край.

```
int sum = 0;
for (int i = 1; ; ++i) {
   sum += i;
   if (i == 100) break;
}
```

Операторът continue пропуска остатъка от текущата итерация. При for се изпълнява актуализацията, след което се проверява условието; при while и do...while управлението преминава към проверката на условието. Внимавайте да не пропуснете необходима актуализация на брояч при while.

```
int sum = 0;
for (int i = 1; i <= 10; ++i) {
   if (i == 7 || i == 8) continue;
   sum += i;  // Сумира всички освен 7 и 8.
}
```

return прекратява текущата функция; return 0; в main() означава успешно приключване, а ненулев код обикновено сигнализира грешка. За прекратяване на цялата програма съществува и std::exit() от <cstdlib>, но в показаните програми използваме return от main() за по-ясна структура.

3. Вложени цикли и надеждно въвеждане

Вложен цикъл е цикъл, разположен в тялото на друг. Вътрешният цикъл се изпълнява за всяка съответна итерация на външния. Обикновено двата цикъла използват различни броячи.

```
for (int row = 1; row <= 3; ++row) {
   for (int col = 1; col <= 4; ++col) {
      std::cout << row * col << " ";
   }
   std::cout << "\n";
}
```

4. Задачи за самостоятелна работа

Задача 5.1. Произведение на положителните числа до въвеждане на 0

Въвежда се последователност от цели числа. Когато се въведе 0, обработката приключва. Пресмята се произведението само на положителните числа. Представени са три решения: с while и първо четене преди цикъла; с безкраен цикъл и break; с проверка на въвеждането в условието на while. При липса на положителни числа това се съобщава изрично. Използвана е проверка срещу препълване.

Задача 5.2. Табулиране на функция със стъпка 0,1

Пресметнете $f(x) = x^4 + 2x^3 - 13x^2 + 8x$ за $0{,}5 \le x \le 2{,}5$ със стъпка 0,1. За да избегнем натрупване на грешки при увеличаване на число с плаваща запетая, използваме целочислен брояч k и x = k / 10.0. Резултатът се подрежда с <iomanip>.

Задача 5.3. Извеждане на символи по ASCII код

Да се покажат символите в избран интервал от ASCII кодове. Стандартната ASCII таблица съдържа 128 кода (0–127); кодовете 32–126 са печатаеми. Вместо зависимото от кодовата таблица старо демо с кодове 200–239, извеждаме главните латински букви с кодове 65–90. Използваме стандартен C++, без conio.h и getch().

Задача 5.4. Ъгли на триъгълник и числена грешка

Въвеждат се три страни a, b, c. С do...while се изисква повторно въвеждане, докато стойностите са положителни и изпълняват неравенствата на триъгълника. Ъглите срещу съответните страни се изчисляват чрез косинусова теорема, която работи и при тъпоъгълни триъгълници. Грешката е |A+B+C−180°|.

Задача 5.5. Меню с математически и тригонометрични функции

Изградете главно меню за log, exp, sqrt, тригонометрични функции и изход. Вложено меню предлага sin, cos, tan и cot. Циклите do...while повтарят менюто до избор за връщане или изход. Проверете областта на дефиниция: log(a) само при a>0, sqrt(a) при a≥0, cot(a) когато sin(a)≠0. Тригонометричните функции приемат радиани.

Задача 5.6. Произведение от n множителя

По зададени x и n≥1 намерете R = ∏(i=1..n) [1 + x^(i+1)/(2i)]. Така се получават множителите ($x^2/2+1$), ($x^3/4+1$), ..., ($x^{n+1}/(2n)+1$). Използвайте for и числова променлива от тип double.

Задача 5.7. Сумиране на степенен ред до точност 10^{-7}

Да се пресметне V = a + (x/a) + $(x/a)^3$ + $(x/a)^5$ + … до достигане на точност 10^{-7}. Нека q=x/a. Редът има крайна сума само при |q|<1; при a=0 изразът е недефиниран. Тъй като всяко следващо събираемо е предишното, умножено по q^2, избягваме повторно извикване на pow(). Използваме do...while и ограничение за броя итерации.

Задача 5.8. Игра „Топло – студено“ с произволно число

Програмата генерира число в диапазона 1..M (M=100), а играчът въвежда предположения до познаване. Запазваме правилото от упражнението: ако предположението е по-малко от числото, се извежда „ТОПЛО“, а ако е по-голямо — „СТУДЕНО“. При вярно предположение се извеждат числото и броят опити. Вместо srand()/rand() използваме <random>.

Задача 5.9. Кубичен корен чрез вложени цикли и итерации

Табулирайте кубичния корен на x за целите x от −10 до 10. Използвайте външен for и вътрешен do...while, който прилага формулата на Нютон $v_{n+1} = v_n + (x/v_n^2 - v_n)/3$ до $|v_{n+1}-v_n|<10^{-5}$. Начално приближение: v=x/3 при x≥1, иначе v=x. За x=0 трябва специална проверка, за да няма деление на нула.

Задача 5.10. Кодиране на латински текст чрез изместване с една буква

Прочетете символи до Enter. Заменете A→B, B→C, …, Z→A и съответно a→b, …, z→a. Цифрите, интервалите и пунктуацията остават непроменени. Използвайте цикъл do...while и std::cin.get() за посимволно четене. Кодира се само латинската азбука.

Задача 5.11. Факториели с вложени цикли и continue

Пресметнете N! за всички N от 5 до 15, като пропуснете 10! и 13!. Външният for обхожда стойностите на N, а вътрешният умножава 1·2·…·N. Операторът continue пропуска зададените две стойности. За 15! е достатъчен unsigned long long.

5. Допълнителни въпроси и задачи за самостоятелна работа

1. Какви са разликите между for, while и do...while? Кой цикъл задължително изпълнява тялото поне веднъж?
2. Колко пъти ще се изпълни тялото на while и do...while, ако условието е false още в началото?
3. За какво служат break и continue? Как се държат при вложени цикли?
4. Какво представлява вложен цикъл? Дайте пример с два брояча.
5. Възможно ли е инициализиране и актуализиране на няколко променливи в for?
6. Напишете програма за въвеждане на числа от 0 до 99 до символ *. Изведете сумата и произведението на въведените числа, броя на четните и нечетните числа и средноаритметичната стойност. Предвидете празна последователност и препълване.
7. Въведете оценки в точки (0–100), като отрицателна стойност приключва въвеждането. Проверявайте валидността на всяка оценка. Изведете броя на оценките, минимум, максимум и броя на оценките в интервал, зададен от потребителя.
8. Табулирайте y(x) за x от −100 до 100 със стъпка 10: при x<0, y=√(x²+1); при x≥0, y=(x+10)/(x−20). Обработете случая x=20, когато функцията не е дефинирана.
9. Напишете програма, която отделя и сумира цифрите на неотрицателно цяло число (unsigned long long).
10. Напишете програма, която брои всички въведени символи до EOF, включително интервали и нови редове.